Projet SDN/NFV sous OpenStack

Conception et déploiement d'une infrastructure cloud sécurisée avec chaînage de fonctions réseau virtualisées

Réalisé par :

Khallouk Aymane

Encadré par :

Pr. Kobbane Abdellatif

Année universitaire : 2025-2026

Table des matières

Introduction **6**

1 Déploiement de l'infrastructure OpenStack **7**
1.1 Installation de MicroStack . . . 7
1.2 Initialisation du nœud contrôleur . . . 7
1.3 Accès à l'interface Horizon . . . 8

2 Conception et implémentation des réseaux SDN **9**
2.1 Architecture réseau . . . 9
2.2 Création des réseaux . . . 9
2.3 Configuration du routage inter-réseaux . . . 10
2.4 Topologie réseau initiale . . . 11

3 Déploiement des instances **14**
3.1 Configuration des groupes de sécurité . . . 14
3.2 Premiers essais d'instances . . . 14
3.3 Déploiement des instances . . . 14
3.4 Tests de connectivité . . . 15
3.5 Topologie avec instances . . . 16

4 Déploiement et configuration des fonctions réseau virtualisées **18**
4.1 Préparation des images et clés SSH . . . 18
4.2 Déploiement des VNF . . . 18
4.3 Configuration du pare-feu (Firewall-VNF) . . . 19
4.3.1 Validation des règles . . . 21
4.4 Configuration du système de détection d'intrusion (IDS-VNF) . . . 21
4.4.1 Connexion à l'IDS . . . 21
4.4.2 Installation de Suricata . . . 22
4.4.3 Règles de détection . . . 23
4.5 Configuration du répartiteur de charge (LB-VNF) . . . 24
4.5.1 Installation de HAProxy . . . 24
4.5.2 Configuration de HAProxy . . . 24
4.5.3 Tests de répartition de charge . . . 25

5 Mise en œuvre du chaînage de services **26**
5.1 Configuration du chaînage sur le pare-feu . . . 26
5.2 Configuration du chaînage sur l'IDS . . . 26
5.3 Validation du chaînage . . . 26

6 Évaluation de la sécurité **28**
6.1 Détection de scan de ports . . . 28
6.2 Détection d'attaque brute force . . . 28
6.3 Détection de ping flood . . . 29

7 Analyse des performances **30**
7.1 Résultats sans sécurité 30
7.2 Résultats avec IDS 30
7.3 Résultats avec répartiteur de charge 31
7.4 Synthèse des performances 31

8 Topologie finale **32**

Conclusion **33**

Table des figures

1 Installation de MicroStack . . . 7
2 Progression de l'initialisation de MicroStack . . . 7
3 Mot de passe administrateur Keystone . . . 8
4 Page de connexion à Horizon . . . 8
5 Création d'un réseau dans Horizon . . . 9
6 Liste des réseaux après création . . . 10
7 Détail des réseaux avec sous-réseaux . . . 10
8 Création du routeur . . . 11
9 Ajout d'interface sur le réseau web-net . . . 11
10 Topologie réseau après configuration du routage . . . 12
11 Topologie détaillée avec tous les réseaux . . . 12
12 Topologie finale avec le routeur . . . 12
13 Règles du groupe de sécurité par défaut . . . 14
14 Instance en erreur . . . 14
15 Premières instances actives . . . 15
16 Configuration DHCP du sous-réseau public-net . . . 15
17 Tests de connectivité depuis le client vers les réseaux internes . . . 16
18 Tests de connectivité depuis l'administrateur . . . 16
19 Topologie avec instances . . . 16
20 Topologie finale avec toutes les instances . . . 17
21 Paire de clés SSH . . . 18
22 Images disponibles . . . 18
23 Liste finale des instances avec VNF . . . 19
24 Connexion à l'instance Firewall-VNF . . . 20
25 Règles iptables sur le pare-feu . . . 20
26 Activation de l'IP forwarding . . . 21
27 Validation des règles du pare-feu . . . 21
28 Connexion à l'instance IDS-VNF . . . 22
29 Installation de Suricata . . . 23
30 Création des règles Suricata . . . 23
31 Ajout des règles de détection avancées . . . 23
32 Installation de HAProxy . . . 24
33 Configuration de HAProxy . . . 24
34 Redémarrage du service HAProxy . . . 25
35 Serveurs HTTP temporaires . . . 25
36 Tests de répartition de charge . . . 25
37 Configuration DNAT sur le pare-feu . . . 26
38 Configuration DNAT sur l'IDS . . . 26
39 Preuve du transit par l'IDS . . . 26
40 Scan NMAP . . . 28
41 Alertes de détection de scan NMAP . . . 28
42 Logs des alertes NMAP . . . 28
43 Attaque Hydra . . . 28
44 Détection de l'attaque brute force . . . 29
45 Ping flood . . . 29
46 Alertes de détection de ping flood . . . 29

47 Mesures sans sécurité 30
48 Mesures avec IDS actif 30
49 Mesures avec répartiteur de charge 31
50 Topologie finale (1/2) 32
51 Topologie finale (2/2) 32

Liste des tableaux

1 Synthèse des performances 31

Introduction

Les paradigmes Software-Defined Networking (SDN) et Network Functions Virtualization (NFV) constituent aujourd'hui les piliers fondamentaux des architectures cloud modernes. Ils permettent de dissocier le plan de contrôle du plan de données et de virtualiser les fonctions réseau traditionnellement exécutées sur des équipements dédiés.

Ce projet vise à concevoir et déployer une infrastructure cloud programmable sécurisée permettant d'héberger des services Web protégés par des fonctions réseau virtualisées. L'infrastructure repose sur OpenStack (MicroStack) et intègre :

- Une segmentation réseau basée sur les principes SDN via Neutron
- Des fonctions réseau virtualisées (pare-feu, système de détection d'intrusion, répartiteur de charge)
- Un chaînage de services (Service Function Chaining) pour orchestrer le trafic à travers les VNF
- Une évaluation des performances et de la résilience de la plateforme

Le présent rapport détaille l'ensemble des étapes de réalisation, depuis le déploiement de l'environnement OpenStack jusqu'aux tests de sécurité et de performance, en s'appuyant sur les captures d'écran effectuées tout au long du projet.

1 Déploiement de l'infrastructure OpenStack

1.1 Installation de MicroStack

L'environnement OpenStack a été déployé à l'aide de MicroStack, une distribution légère de OpenStack empaquetée sous forme de snap. L'installation de la version bêta (Ussuri) a été réalisée.

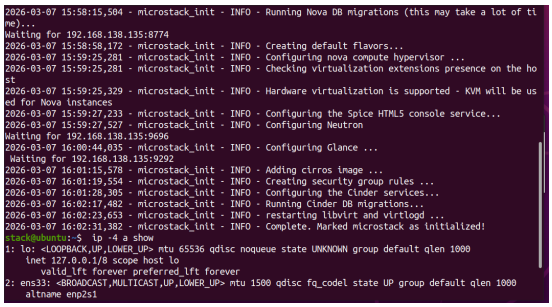
```
stack@ubuntu:~$ sudo snap install microstack --beta
microstack (beta) ussuri from Canonical✓ installed
stack@ubuntu:~$ sudo microstack init --auto --control
sudo: microstack: command not found
stack@ubuntu:~$ sudo /snap/bin/microstack init --auto --control
2026-03-07 15:56:34,628 - microstack_init - INFO - Configuring clustering ...
2026-03-07 15:56:34,801 - microstack_init - INFO - Setting up as a control node.
2026-03-07 15:56:37,603 - microstack_init - INFO - Generating TLS Certificate and Key
2026-03-07 15:56:38,312 - microstack_init - INFO - Configuring networking ...
2026-03-07 15:56:43,724 - microstack_init - INFO - Opening horizon dashboard up to *
2026-03-07 15:56:44,621 - microstack_init - INFO - Waiting for RabbitMQ to start ...
Waiting for 192.168.138.135:5672
2026-03-07 15:56:50,887 - microstack_init - INFO - RabbitMQ started!
2026-03-07 15:56:50,888 - microstack_init - INFO - Configuring RabbitMQ ...
2026-03-07 15:56:51,694 - microstack_init - INFO - RabbitMQ Configured!
2026-03-07 15:56:51,716 - microstack_init - INFO - Waiting for MySQL server to start ...
Waiting for 192.168.138.135:3306
2026-03-07 15:56:56,990 - microstack_init - INFO - Mysql server started! Creating databases ...
```

FIGURE 1 – Installation de MicroStack

La capture montre l'installation réussie du snap microstack en version bêta. On observe que la commande microstack n'est pas directement accessible, nécessitant l'utilisation du chemin complet /snap/bin/microstack par la suite.

1.2 Initialisation du nœud contrôleur

L'initialisation en mode contrôleur a été effectuée avec la commande `sudo /snap/bin/microstack init -auto -control`. Le processus d'initialisation est présenté ci-dessous.



(a) Configuration des services



(b) Interfaces réseau après initialisation

FIGURE 2 – Progression de l'initialisation de MicroStack

La première capture montre le déroulement de l'initialisation avec la configuration des services Nova, Neutron, Glance et Cinder. La seconde capture présente les interfaces réseau

créées, notamment l'interface br-ex avec l'adresse 10.20.20.1/24 qui servira de passerelle pour le réseau externe.

1.3 Accès à l'interface Horizon

Le mot de passe administrateur a été récupéré via la commande snap.

```
stack@ubuntu:~$ sudo snap get microstack config.credentials.keystone-password
eKr4Cxm6HD4chFJaInw6Q5igMzNRA1GE
```

FIGURE 3 – Mot de passe administrateur Keystone

La commande snap get permet de récupérer le mot de passe généré automatiquement pour l'utilisateur admin. Ce mot de passe sera utilisé pour se connecter à l'interface web Horizon.

L'interface web Horizon a été accessible à l'adresse `https://10.20.20.1`.

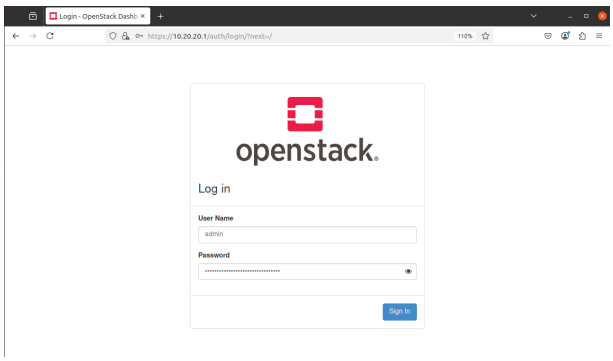


FIGURE 4 – Page de connexion à Horizon

La page de connexion à Horizon est accessible via HTTPS. L'utilisateur admin peut s'authentifier avec le mot de passe récupéré précédemment pour accéder à l'interface d'administration graphique d'OpenStack.

2 Conception et implémentation des réseaux SDN

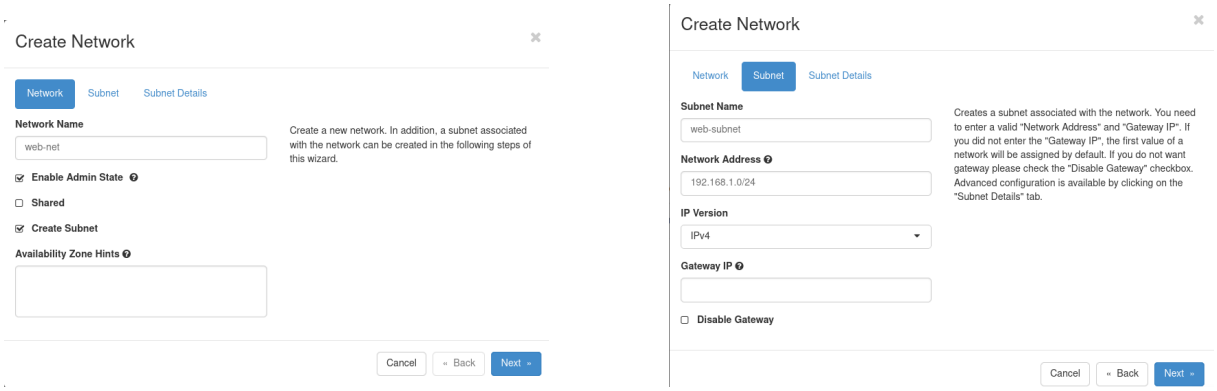
2.1 Architecture réseau

Conformément aux spécifications du projet, l'infrastructure réseau a été conçue avec quatre réseaux isolés :

- **public-net** (10.20.20.0/24) : réseau externe avec accès à Internet
- **web-net** (192.168.1.0/24) : réseau dédié aux serveurs Web
- **admin-net** (192.168.2.0/24) : réseau d'administration
- **security-net** (192.168.3.0/24) : réseau pour les fonctions réseau virtualisées

2.2 Création des réseaux

La création des réseaux a été réalisée via l'interface graphique Horizon. Les captures suivantes illustrent le processus pour le réseau `web-net`.



(a) Création du réseau web-net (b) Définition du sous-réseau

FIGURE 5 – Création d'un réseau dans Horizon

La première capture montre la création du réseau web-net avec l'option "Create Subnet" activée. La seconde capture présente la configuration du sous-réseau avec l'adresse 192.168.1.0/24 en IPv4.

L'ensemble des réseaux créés est présenté ci-après.

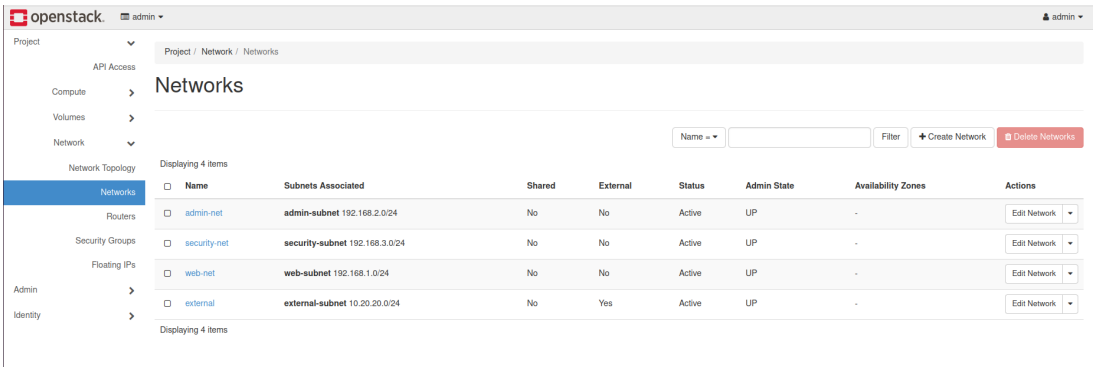


FIGURE 6 – Liste des réseaux après création

On distingue les quatre réseaux créés : web-net, admin-net, security-net et public-net. Chaque réseau possède son sous-réseau associé et est en état "Active".

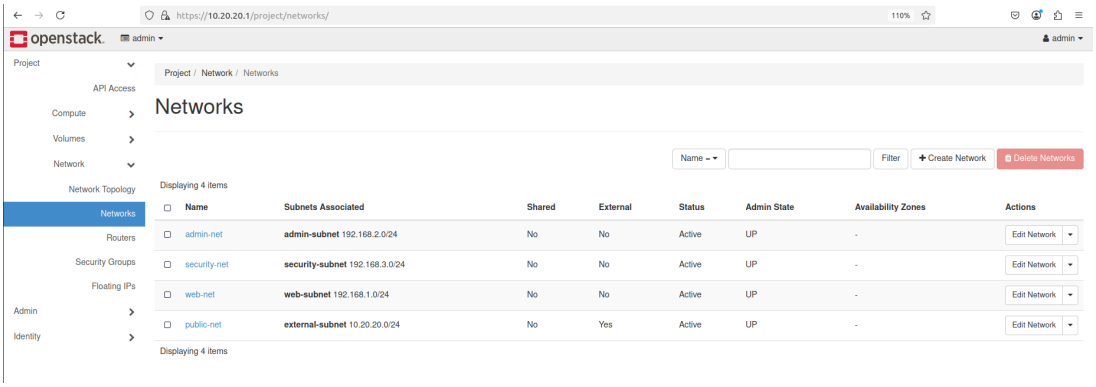


FIGURE 7 – Détail des réseaux avec sous-réseaux

Cette vue détaillée montre les sous-réseaux associés à chaque réseau. On note que public-net est marqué comme "External" et sera utilisé pour l'accès depuis l'extérieur.

2.3 Configuration du routage inter-réseaux

Un routeur nommé `Router-OpenStack` a été instancié pour assurer la connectivité entre les différents réseaux.

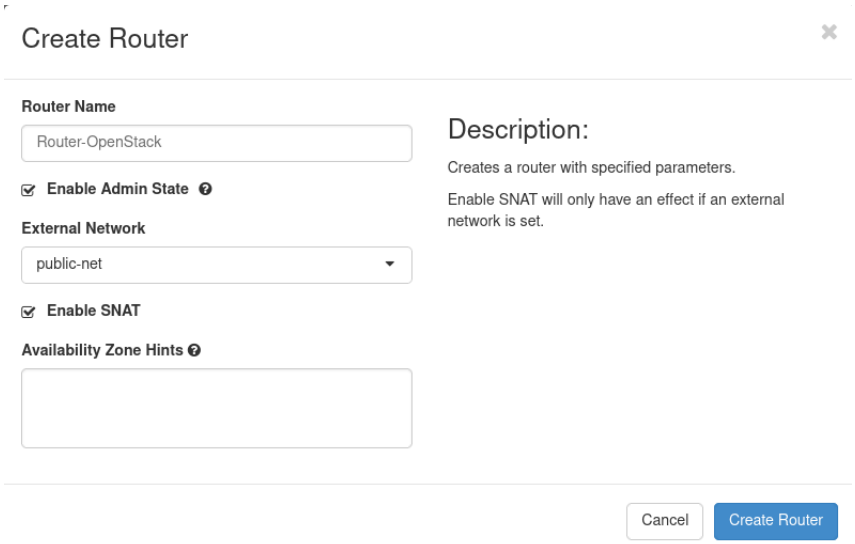


FIGURE 8 – Création du routeur

Le routeur est créé avec le réseau externe public-net comme passerelle par défaut. L'option "Enable SNAT" est activée pour permettre la traduction d'adresses vers l'extérieur.

Des interfaces ont été ajoutées pour chaque réseau privé.

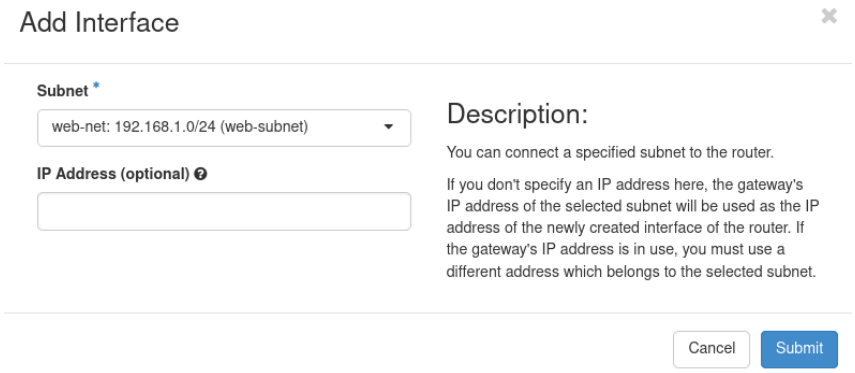
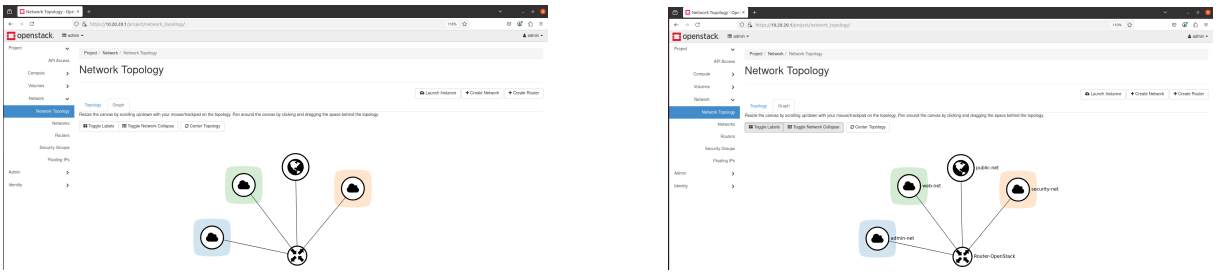


FIGURE 9 – Ajout d'interface sur le réseau web-net

L'ajout d'une interface sur le réseau web-net permet de connecter ce réseau au routeur. L'adresse IP de la passerelle sera automatiquement attribuée (première adresse du sous-réseau).

2.4 Topologie réseau initiale

La topologie obtenue après configuration du routeur est présentée ci-dessous.



(a) Vue initiale (b) Vue avec labels activés

FIGURE 10 – Topologie réseau après configuration du routage

Les deux vues de la topologie montrent le routeur central connecté aux quatre réseaux. On distingue clairement les passerelles (adresses .1) sur chaque réseau.

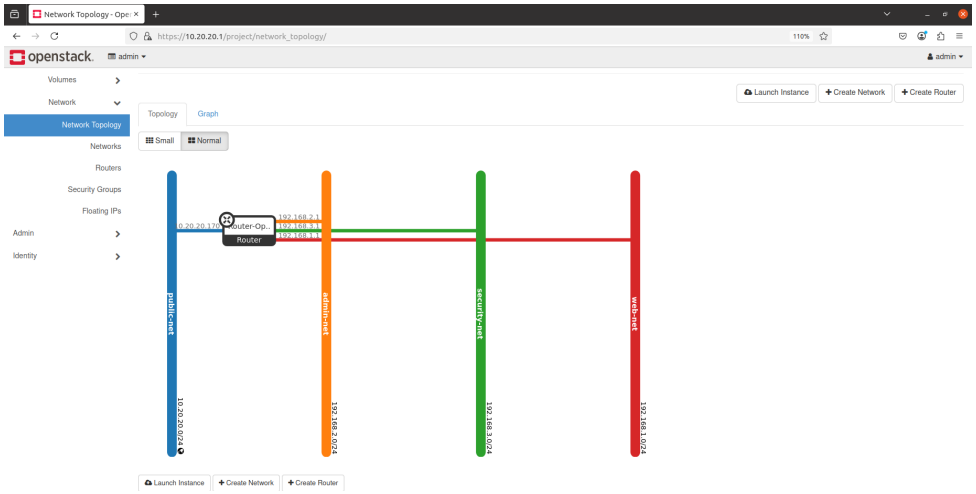


FIGURE 11 – Topologie détaillée avec tous les réseaux

Cette vue détaillée permet de visualiser l'ensemble des réseaux et leurs interconnexions via le routeur unique.

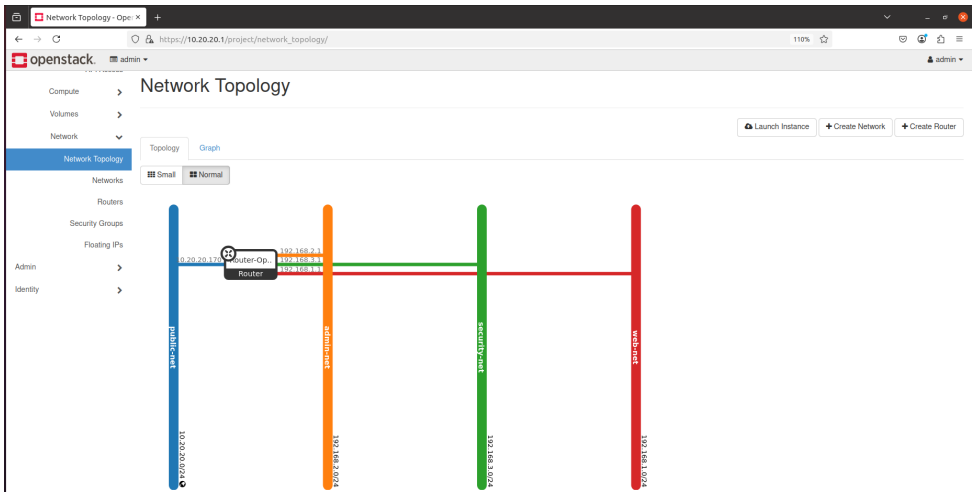


FIGURE 12 – Topologie finale avec le routeur

La topologie montre clairement le routeur central avec ses interfaces sur chaque réseau. Les adresses IP des passerelles sont visibles (10.20.20.1, 192.168.1.1, 192.168.2.1, 192.168.3.1).

3 Déploiement des instances

3.1 Configuration des groupes de sécurité

Avant le déploiement, les règles du groupe de sécurité par défaut ont été vérifiées.

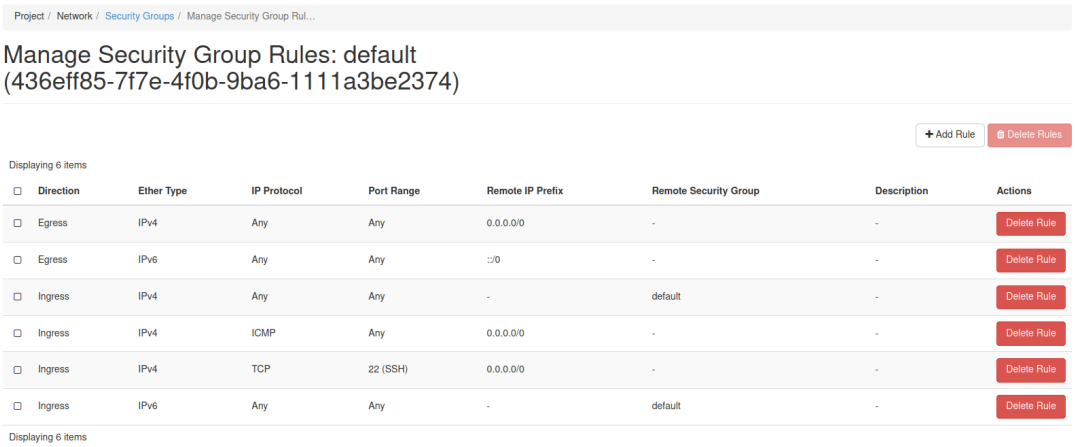


FIGURE 13 – Règles du groupe de sécurité par défaut

Les règles par défaut autorisent tout le trafic egress, le trafic ingress du même groupe, ainsi que ICMP et SSH (port 22) depuis n'importe quelle source. Ces règles seront suffisantes pour nos tests initiaux.

3.2 Premiers essais d'instances

Un premier essai de lancement d'instance a rencontré une erreur.

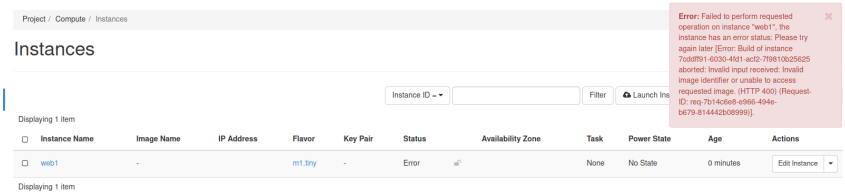


FIGURE 14 – Instance en erreur

L'instance web1 est en état "Error", probablement dû à un problème de ressources ou de configuration réseau. Ce problème sera résolu par la suite.

3.3 Déploiement des instances

Après résolution, les instances ont été déployées avec succès.

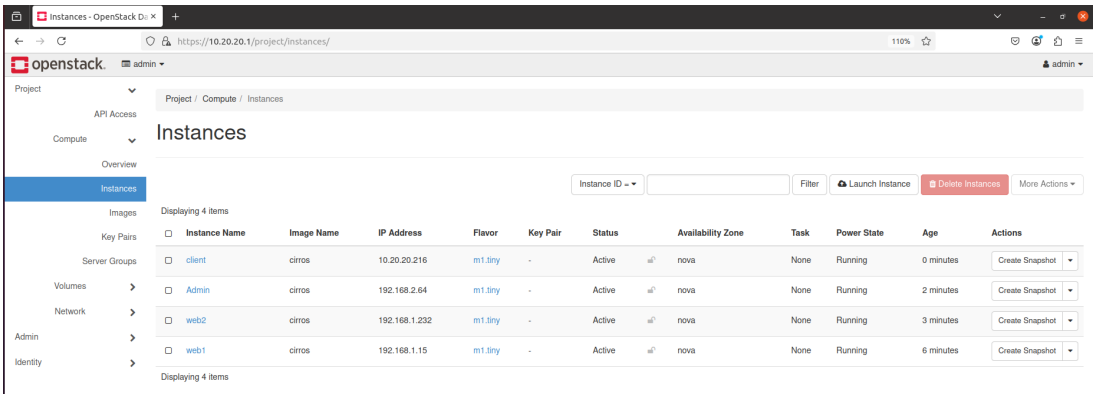


FIGURE 15 – Premières instances actives

Quatre instances sont maintenant actives : client, Admin, web1 et web2. Elles disposent d'adresses IP privées sur leurs réseaux respectifs.

3.4 Tests de connectivité

Des tests de ping ont été réalisés pour valider la connectivité entre le client et les réseaux internes.

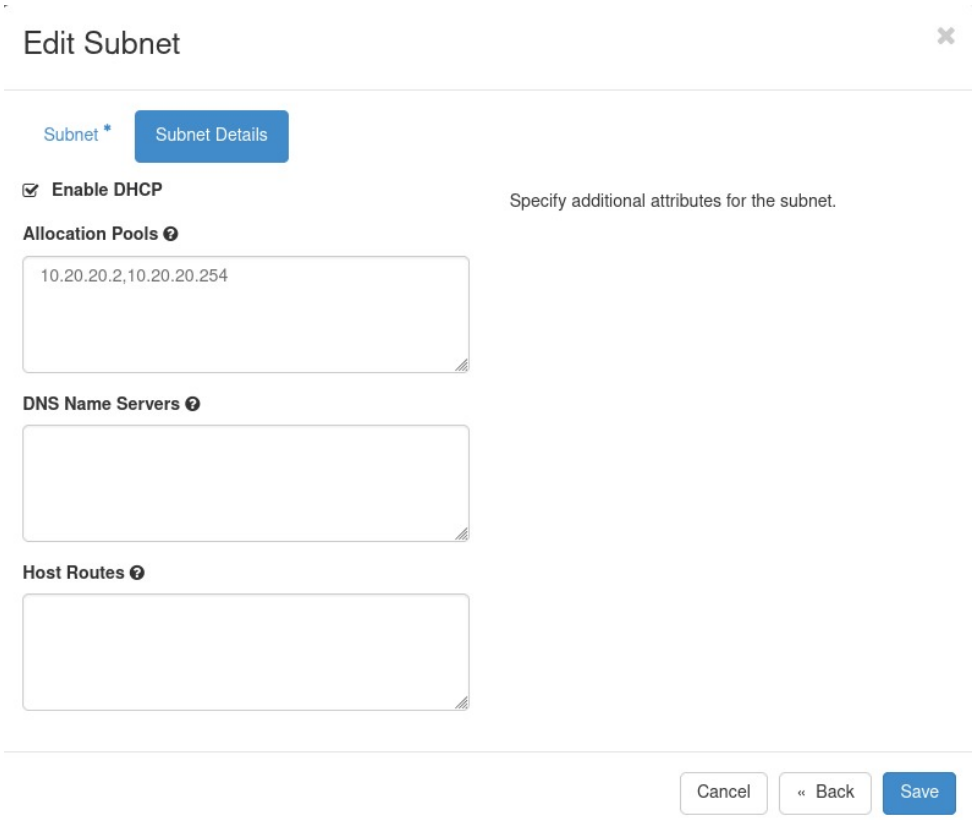
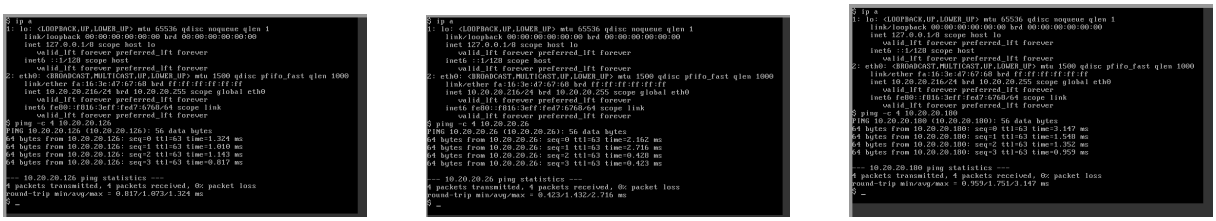


FIGURE 16 – Configuration DHCP du sous-réseau public-net

La configuration du sous-réseau public-net montre que DHCP est activé avec un pool d'adresses allant de 10.20.20.2 à 10.20.20.254. Le serveur DNS 1.1.1.1 est configuré,

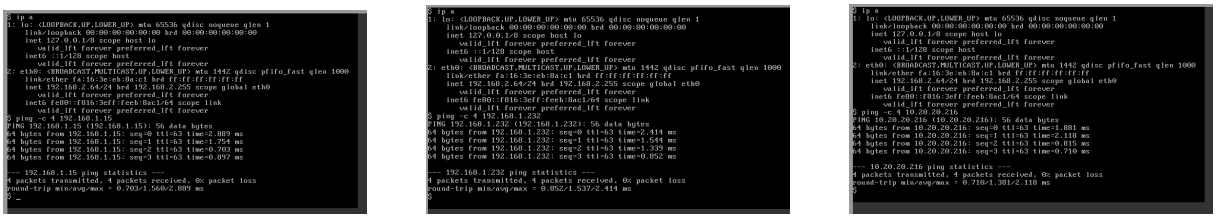
ainsi qu'une route statique vers le réseau interne 192.168.1.0/24.



(a) Client → web1 (b) Client → web2 (c) Client → Admin

FIGURE 17 – Tests de connectivité depuis le client vers les réseaux internes

Les pings depuis l'instance client (10.20.20.216) vers les adresses IP flottantes des instances internes réussissent tous : web1 (10.20.20.126), web2 (10.20.20.26) et Admin (10.20.20.180). Ces tests confirment le bon routage via le routeur OpenStack et l'accès du client aux réseaux internes.



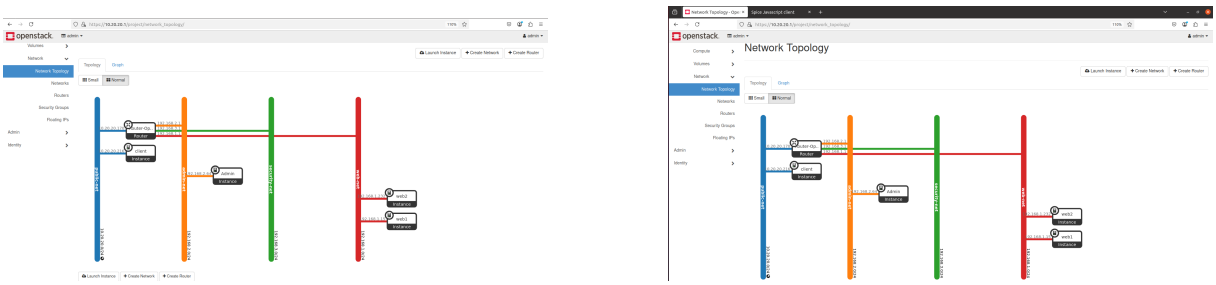
(a) Admin → web1 (b) Admin → web2 (c) Admin → client

FIGURE 18 – Tests de connectivité depuis l'administrateur

Depuis l'instance Admin (192.168.2.64), les pings vers web1 (192.168.1.15), web2 (192.168.1.232) et le client (10.20.20.216) réussissent également, validant la connectivité bidirectionnelle entre tous les réseaux.

3.5 Topologie avec instances

La topologie réseau après déploiement des instances est visible ci-dessous.



(a) Vue d'ensemble (b) Détail des connexions

FIGURE 19 – Topologie avec instances

La topologie montre maintenant les instances connectées à leurs réseaux respectifs. On distingue clairement les quatre instances : client (sur public-net), Admin (sur admin-net), web1 et web2 (sur web-net).

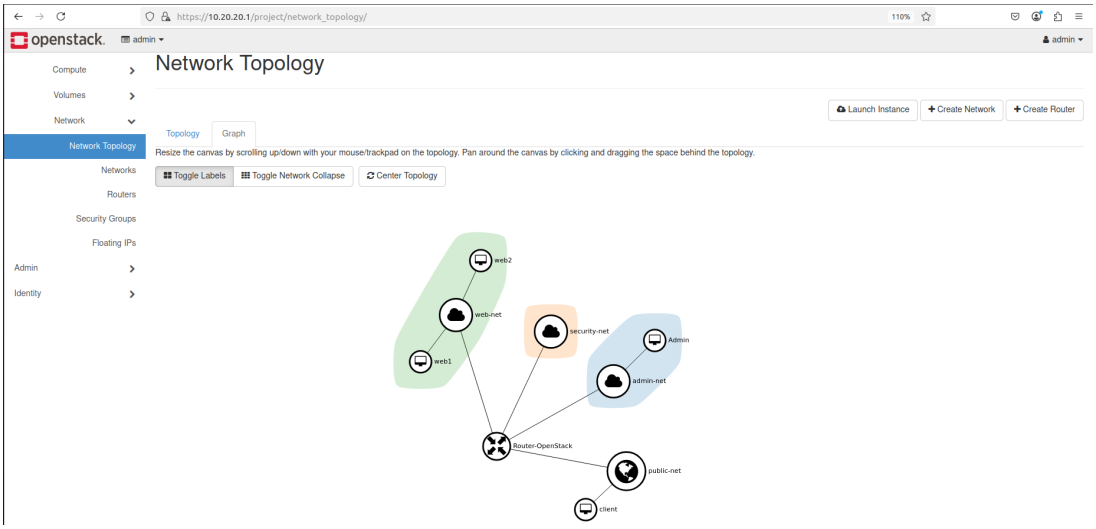


FIGURE 20 – Topologie finale avec toutes les instances

Cette vue offre une vision complète de l'infrastructure avec les quatre réseaux, le routeur central et les quatre instances déployées.

4 Déploiement et configuration des fonctions réseau virtualisées

4.1 Préparation des images et clés SSH

La paire de clés SSH `cle-projet` a été importée pour permettre l'accès aux instances Ubuntu.

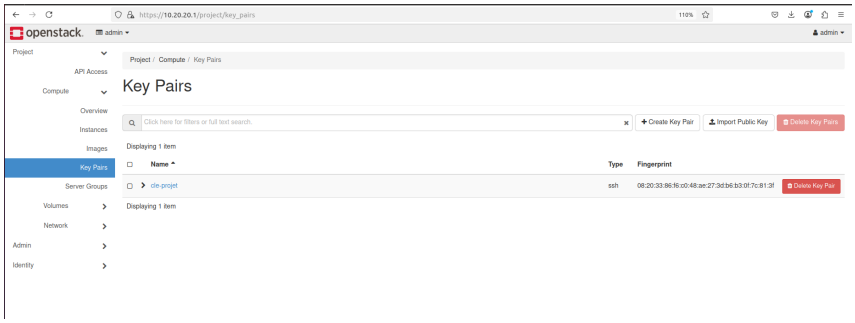


FIGURE 21 – Paire de clés SSH

La clé SSH "cle-projet" a été importée dans OpenStack. Elle permettra de se connecter aux instances Ubuntu sans mot de passe.

Les images disponibles sont présentées ci-dessous.



FIGURE 22 – Images disponibles

Deux images sont disponibles : cirros (image légère pour les tests) et Ubuntu-20.04 (pour les VNF). Ces images ont été téléchargées préalablement dans Glance.

4.2 Déploiement des VNF

Les VNF Firewall, IDS et LB ont été déployées.

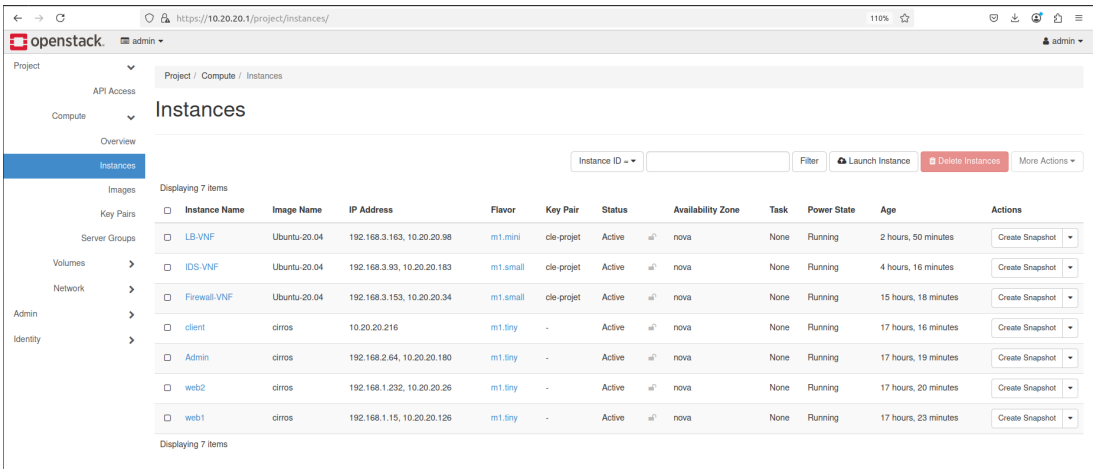


FIGURE 23 – Liste finale des instances avec VNF

Les trois VNF (Firewall-VNF, IDS-VNF, LB-VNF) sont maintenant déployées avec succès. Elles utilisent l'image Ubuntu-20.04 et le flavor m1.small. Chaque VNF possède une adresse IP privée sur le réseau security-net (192.168.3.0/24) et une adresse IP flottante associée.

4.3 Configuration du pare-feu (Firewall-VNF)

La connexion à l'instance Firewall-VNF a été établie.

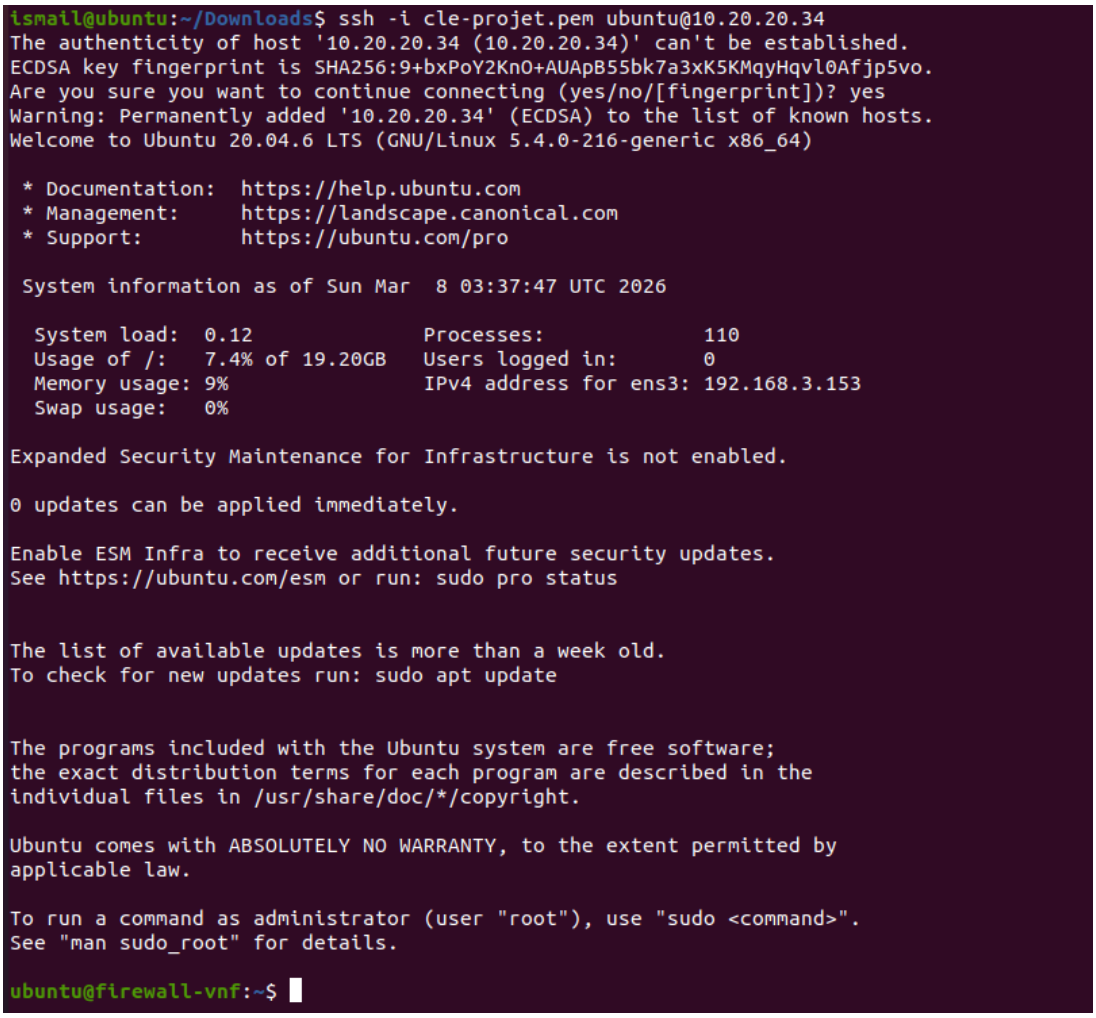


FIGURE 24 – Connexion à l'instance Firewall-VNF

La connexion SSH à l'instance Firewall-VNF via son adresse IP flottante 10.20.20.34 est établie avec succès en utilisant la clé privée cle-projet.pem.

Les règles iptables suivantes ont été implémentées.

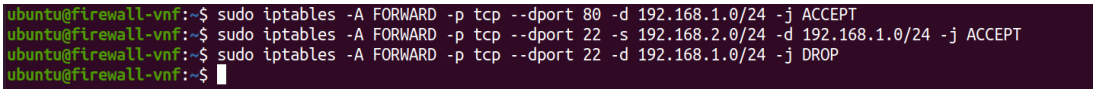


FIGURE 25 – Règles iptables sur le pare-feu

Les règles iptables implémentées autorisent : (1) le trafic HTTP (port 80) vers les serveurs Web, (2) le trafic SSH depuis le réseau d'administration (192.168.2.0/24) vers les serveurs Web, et (3) bloquent tout autre trafic SSH vers les serveurs Web.

L'activation de l'IP forwarding est montrée ci-après.

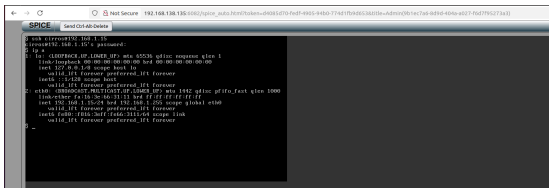
```
ubuntu@firewall-vnf:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@firewall-vnf:~$ sudo iptables -A FORWARD -p tcp --dport 80 -j ACCEPT
ubuntu@firewall-vnf:~$ sudo iptables -A FORWARD -p tcp --dport 22 -s 192.168.2.64 -d 192.168.1.15 -j ACCEPT
ubuntu@firewall-vnf:~$ sudo iptables -A FORWARD -p tcp --dport 22 -d 192.168.1.15 -j DROP
ubuntu@firewall-vnf:~$
```

FIGURE 26 – Activation de l'IP forwarding

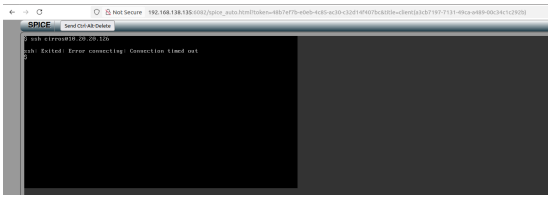
La commande sysctl active le forwarding IP, nécessaire pour que le pare-feu puisse router le trafic entre ses interfaces réseau.

4.3.1 Validation des règles

Les tests suivants ont confirmé le bon fonctionnement des règles.



(a) SSH administrateur → web1 réussi



(b) SSH client → web1 échoué

FIGURE 27 – Validation des règles du pare-feu

Les tests confirment l'application correcte des règles : l'administrateur (192.168.2.64) peut se connecter en SSH à web1 (192.168.1.15), tandis que le client (10.20.20.216) reçoit un timeout, conformément à la politique de sécurité.

4.4 Configuration du système de détection d'intrusion (IDS-VNF)

4.4.1 Connexion à l'IDS

La connexion à l'IDS a été établie.

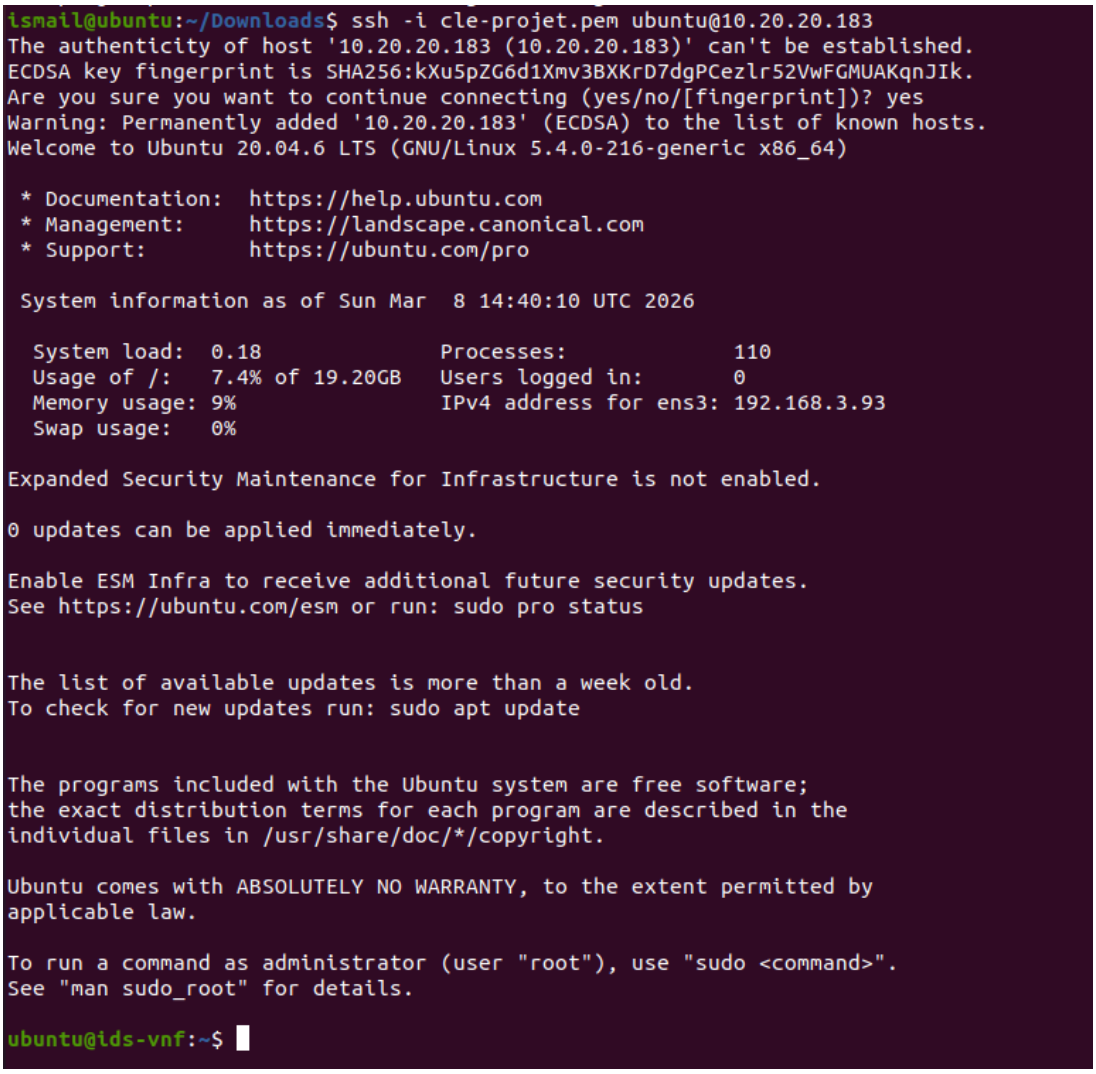


FIGURE 28 – Connexion à l'instance IDS-VNF

La connexion SSH à l'instance IDS-VNF via son adresse IP flottante 10.20.20.183 est établie avec succès.

4.4.2 Installation de Suricata

L'installation de Suricata est illustrée ci-dessous.

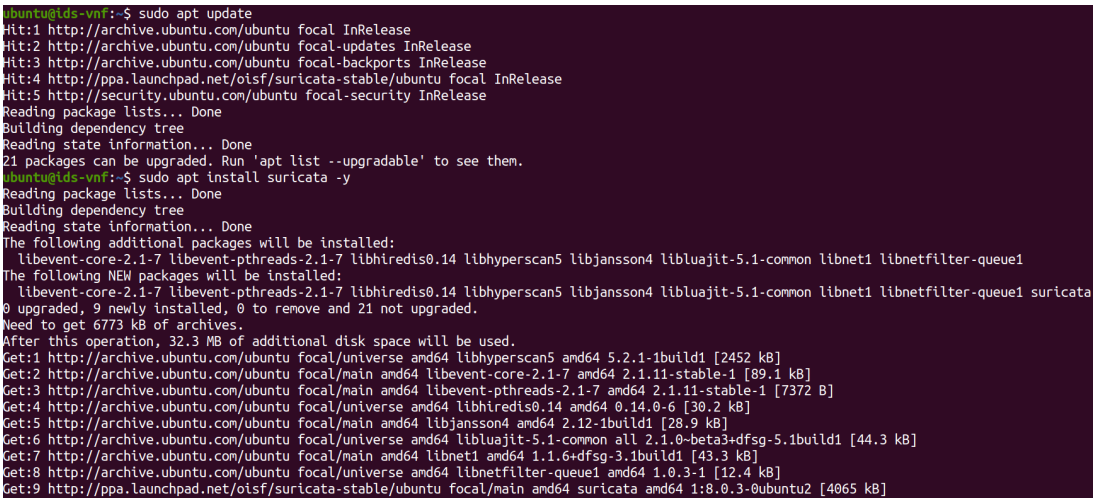


FIGURE 29 – Installation de Suricata

L'installation de Suricata via apt installe également les dépendances nécessaires (libjansson4, libnet1, libnetfilter-queue1, etc.). Le système de détection d'intrusion est maintenant prêt à être configuré.

4.4.3 Règles de détection

La création des règles personnalisées est montrée ci-après.

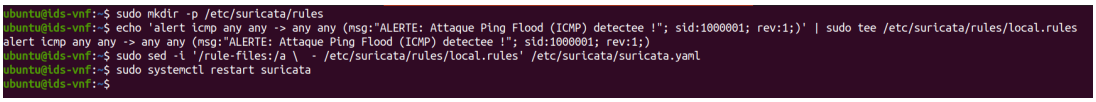


FIGURE 30 – Création des règles Suricata

Une première règle simple est créée pour détecter le trafic ICMP (ping). Le répertoire /etc/suricata/rules est créé pour stocker les règles personnalisées, et le fichier de configuration est modifié pour inclure ces règles.

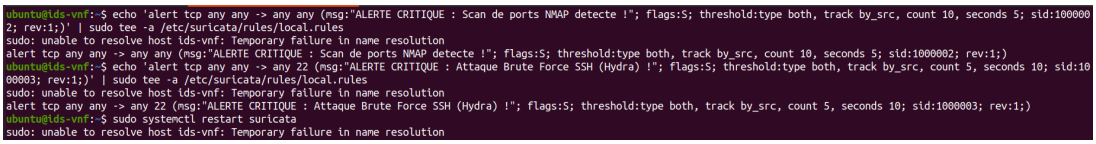


FIGURE 31 – Ajout des règles de détection avancées

Des règles plus sophistiquées sont ajoutées : (1) détection de scan de ports NMAP basée sur un seuil de 10 paquets SYN en 5 secondes, et (2) détection d'attaque brute force SSH basée sur un seuil de 5 paquets SYN en 10 secondes.

4.5 Configuration du répartiteur de charge (LB-VNF)

4.5.1 Installation de HAProxy

L'installation de HAProxy est présentée ci-dessous.

```
ubuntu@lb-vnf:~$ sudo apt install haproxy -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  liblua5.3-0
Suggested packages:
  vim-haproxy haproxy-doc
The following NEW packages will be installed:
  haproxy liblua5.3-0
0 upgraded, 2 newly installed, 0 to remove and 21 not upgraded.
Need to get 1658 kB of archives.
After this operation, 3811 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal/main amd64 liblua5.3-0 amd64 5.3.3-1.1ubuntu2 [116 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 haproxy amd64 2.0.33-0ubuntu0.1 [1542 kB]
Fetched 1658 kB in 1s (1606 kB/s)
Selecting previously unselected package liblua5.3-0:amd64.
(Reading database ... 64241 files and directories currently installed.)
Preparing to unpack .../liblua5.3-0_5.3.3-1.1ubuntu2_amd64.deb ...
Unpacking liblua5.3-0:amd64 (5.3.3-1.1ubuntu2) ...
Selecting previously unselected package haproxy.
Preparing to unpack .../haproxy_2.0.33-0ubuntu0.1_amd64.deb ...
Unpacking haproxy (2.0.33-0ubuntu0.1) ...
Setting up liblua5.3-0:amd64 (5.3.3-1.1ubuntu2) ...
Setting up haproxy (2.0.33-0ubuntu0.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/haproxy.service → /lib/systemd/system/haproxy.service.
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.18) ...
Processing triggers for rsyslog (8.2001.0-1ubuntu1.3) ...
Processing triggers for systemd (245.4-4ubuntu3.24) ...
```

FIGURE 32 – Installation de HAProxy

HAProxy est installé avec ses dépendances. Le service est automatiquement configuré pour démarrer au boot.

4.5.2 Configuration de HAProxy

La configuration de HAProxy est montrée ci-après.

```
 GNU nano 4.8                    /etc/haproxy/haproxy.cfg                    Modified

defaults
        log     global
        mode    http
        option  httplog
        option  dontlognull
        timeout connect 5000
        timeout client  50000
        timeout server  50000
        errorfile 400 /etc/haproxy/errors/400.http
        errorfile 403 /etc/haproxy/errors/403.http
        errorfile 408 /etc/haproxy/errors/408.http
        errorfile 500 /etc/haproxy/errors/500.http
        errorfile 502 /etc/haproxy/errors/502.http
        errorfile 503 /etc/haproxy/errors/503.http
        errorfile 504 /etc/haproxy/errors/504.http

frontend mon_site_web
    bind *:80
    default_backend mes_serveurs_web

backend mes_serveurs_web
    balance roundrobin
    server web1 192.168.1.15:80 check
    server web2 192.168.1.232:80 check
```

FIGURE 33 – Configuration de HAProxy

Le fichier de configuration est modifié pour définir un frontend écoutant sur le port 80 et un backend avec deux serveurs web1 et web2 en mode round-robin. Les paramètres de timeout sont configurés pour éviter les timeouts prématurés.

Le redémarrage du service est illustré ci-dessous.

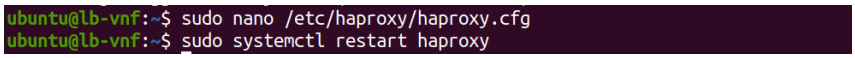


FIGURE 34 – Redémarrage du service HAProxy

Après modification de la configuration, le service HAProxy est redémarré pour prendre en compte les nouveaux paramètres.

4.5.3 Tests de répartition de charge

Des serveurs HTTP simples ont été lancés sur les instances web1 et web2.



(a) Serveur HTTP sur web1



(b) Serveur HTTP sur web2

FIGURE 35 – Serveurs HTTP temporaires

Des serveurs HTTP simples sont lancés sur web1 et web2 en utilisant netcat. Chaque serveur répond avec un message distinct ("Bienvenue sur WEB 1" ou "WEB 2") pour identifier quel serveur a traité la requête.

Les tests avec curl ont confirmé le fonctionnement du round-robin.

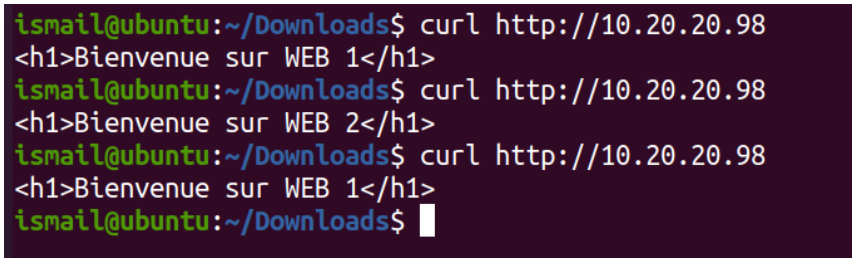


FIGURE 36 – Tests de répartition de charge

Les requêtes curl successives vers l'adresse IP flottante du LB (10.20.20.98) alternent entre les réponses de web1 et web2, démontrant le fonctionnement correct du round-robin.

5 Mise en œuvre du chaînage de services

5.1 Configuration du chaînage sur le pare-feu

La configuration DNAT sur le pare-feu est présentée ci-dessous.

```
ubuntu@firewall-vnf:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@firewall-vnf:~$ sudo iptables -t nat -A PREROUTING -p tcp --dport 8001 -j DNAT --to-destination 192.168.3.93:8001
ubuntu@firewall-vnf:~$ sudo iptables -t nat -A PREROUTING -p tcp --dport 8002 -j DNAT --to-destination 192.168.3.93:8002
ubuntu@firewall-vnf:~$ sudo iptables -t nat -A POSTROUTING -j MASQUERADE
ubuntu@firewall-vnf:~$
```

FIGURE 37 – Configuration DNAT sur le pare-feu

Le pare-feu est configuré pour rediriger le trafic destiné aux ports 8001 et 8002 vers l'IDS (192.168.3.93) sur les mêmes ports. La règle MASQUERADE assure la traduction d'adresse pour le trafic sortant.

5.2 Configuration du chaînage sur l'IDS

La configuration DNAT sur l'IDS est présentée ci-après.

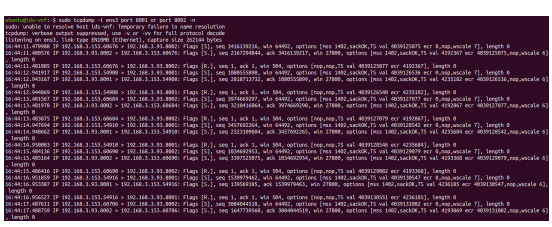
```
ubuntu@ids-vnf:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@ids-vnf:~$ sudo iptables -t nat -A PREROUTING -p tcp --dport 8001 -j DNAT --to-destination 192.168.1.15:80
ubuntu@ids-vnf:~$ sudo iptables -t nat -A PREROUTING -p tcp --dport 8002 -j DNAT --to-destination 192.168.1.232:80
ubuntu@ids-vnf:~$ sudo iptables -t nat -A POSTROUTING -j MASQUERADE
ubuntu@ids-vnf:~$
```

FIGURE 38 – Configuration DNAT sur l'IDS

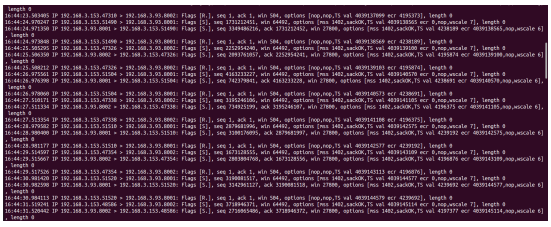
L'IDS redirige à son tour le trafic des ports 8001 et 8002 vers les serveurs web réels : port 8001 vers web1 :80, port 8002 vers web2 :80. La chaîne de services est maintenant complète.

5.3 Validation du chaînage

Une capture tcpdump sur l'IDS a permis de confirmer le transit du trafic.



(a) Capture tcpdump (1)



(b) Capture tcpdump (2)

FIGURE 39 – Preuve du transit par l'IDS

La capture tcpdump sur l'IDS montre clairement le trafic sur les ports 8001 et 8002, prouvant que les paquets transitent bien par l'IDS avant d'atteindre les serveurs web. Les paquets SYN sont visibles, confirmant l'établissement des connexions TCP.

6 Évaluation de la sécurité

6.1 Détection de scan de ports

Un scan NMAP a été lancé depuis le client.

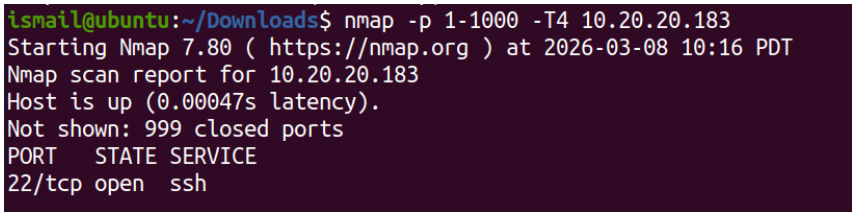


FIGURE 40 – Scan NMAP

Un scan NMAP est lancé depuis la machine cliente vers l'adresse IP flottante de l'IDS (10.20.20.183) pour détecter les ports ouverts.

Suricata a généré les alertes correspondantes.

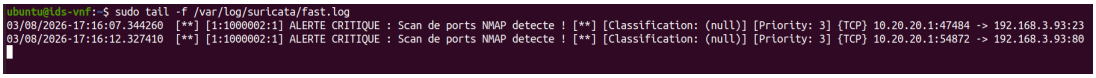


FIGURE 41 – Alertes de détection de scan NMAP

Suricata détecte immédiatement le scan et génère des alertes. Les messages "ALERTE CRITIQUE : Scan de ports NMAP detecte" apparaissent dans le log fast.log.

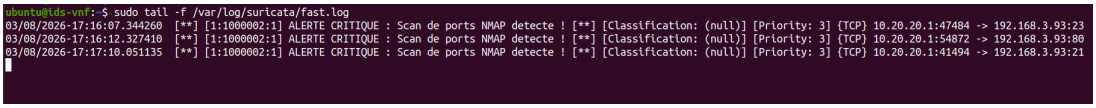


FIGURE 42 – Logs des alertes NMAP

Les logs détaillés montrent les différentes tentatives de connexion sur les ports 23, 80, 21, confirmant que le scan NMAP a bien été détecté et enregistré par Suricata.

6.2 Détection d'attaque brute force

Une attaque Hydra a été simulée.

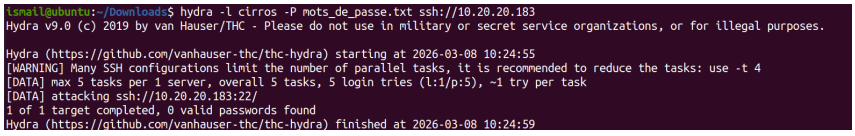


FIGURE 43 – Attaque Hydra

L'outil Hydra est utilisé pour simuler une attaque brute force sur le service SSH de l'IDS. L'attaque tente 5 combinaisons login/mot de passe.

Les alertes ont été enregistrées.

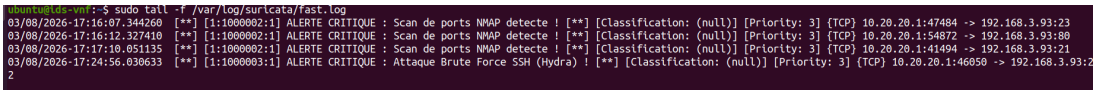


FIGURE 44 – Détection de l'attaque brute force

Suricata détecte l'attaque brute force grâce à la règle basée sur le seuil de connexions SYN. L'alerte "Attaque Brute Force SSH (Hydra)" est générée et enregistrée dans les logs.

6.3 Détection de ping flood

Un ping flood a été lancé.

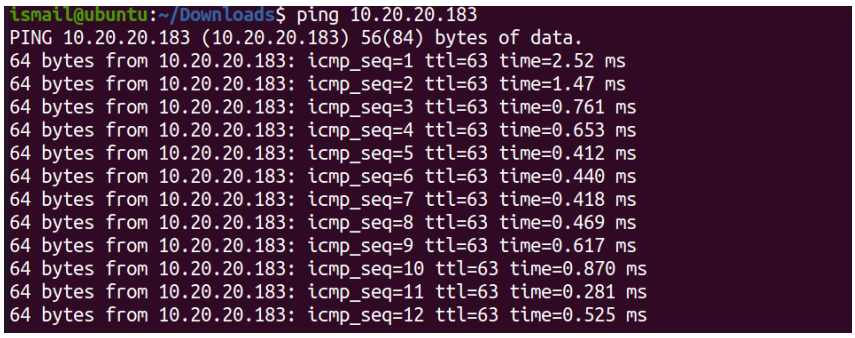


FIGURE 45 – Ping flood

Une série de pings est envoyée depuis le client vers l'IDS pour simuler un trafic ICMP anormal.

Les alertes ICMP ont été générées.

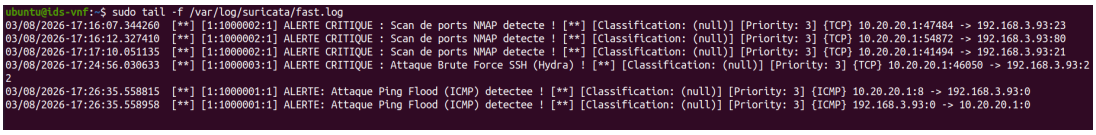


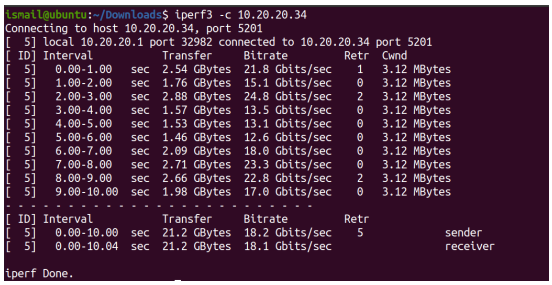
FIGURE 46 – Alertes de détection de ping flood

Les alertes "Attaque Ping Flood (ICMP) detectee" apparaissent dans les logs, confirmant que Suricata détecte également ce type d'attaque basée sur un grand nombre de paquets ICMP.

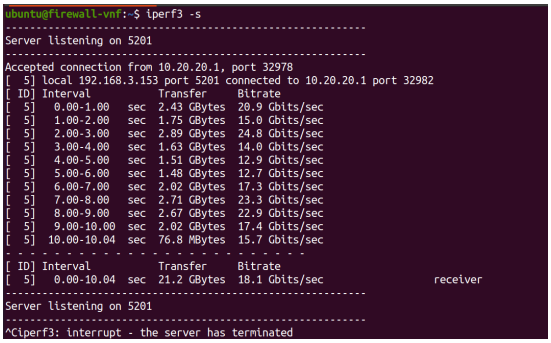
7 Analyse des performances

7.1 Résultats sans sécurité

Les mesures sans sécurité sont présentées ci-dessous.



(a) Client vers pare-feu



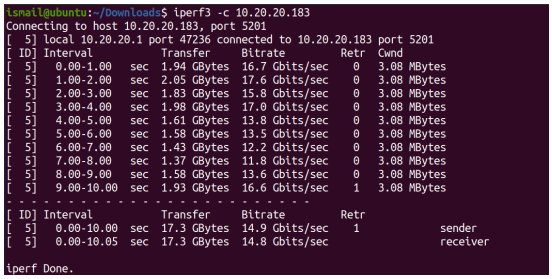
(b) Côté serveur (pare-feu)

FIGURE 47 – Mesures sans sécurité

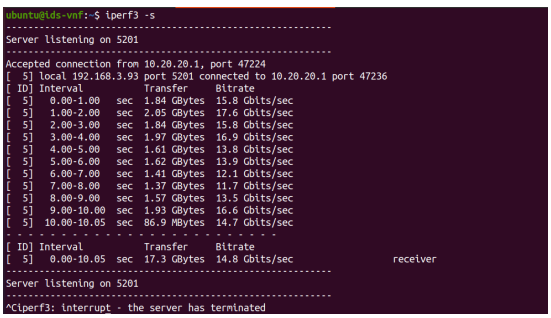
Les mesures iperf entre le client et le pare-feu (sans inspection de trafic) montrent un débit moyen d'environ 18,1 Gbits/s. Cette mesure servira de référence pour évaluer l'impact des fonctions de sécurité.

7.2 Résultats avec IDS

Les mesures avec IDS actif sont présentées ci-après.



(a) Client vers IDS



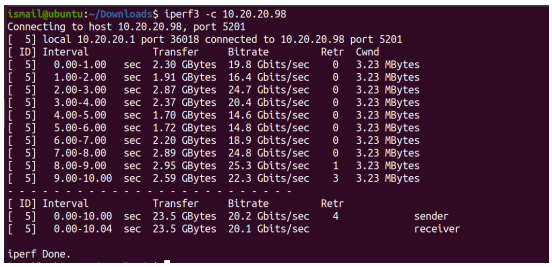
(b) Côté serveur (IDS)

FIGURE 48 – Mesures avec IDS actif

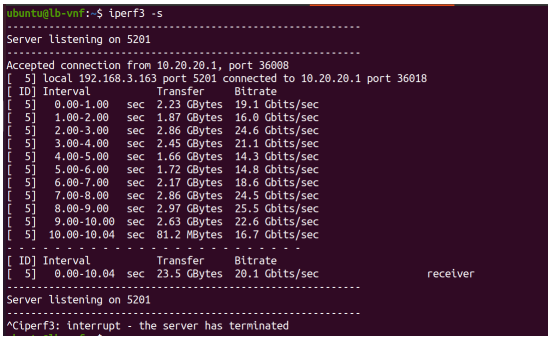
Avec l'IDS actif (Suricata inspectant le trafic), le débit moyen chute à 14,8 Gbits/s, soit une réduction d'environ 18%. Cette baisse est due au temps de traitement nécessaire à l'inspection des paquets par l'IDS.

7.3 Résultats avec répartiteur de charge

Les mesures avec répartiteur de charge sont présentées ci-dessous.



(a) Client vers LB



(b) Côté serveur (LB)

FIGURE 49 – Mesures avec répartiteur de charge

Avec le répartiteur de charge HAProxy, le débit atteint 20,1 Gbits/s, soit une amélioration de 11% par rapport à la référence. Cette augmentation s'explique par la répartition optimale de la charge entre les serveurs et l'efficacité du traitement de HAProxy.

7.4 Synthèse des performances

Le tableau ci-dessous présente une synthèse comparative des performances mesurées.

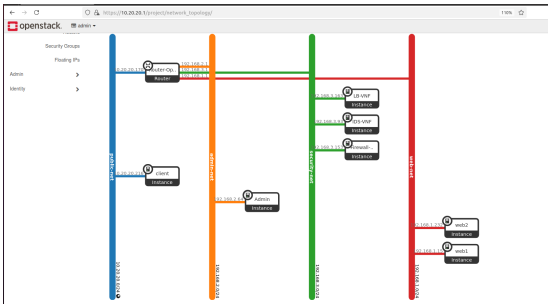
Configuration	**Débit moyen (Gbits/s)**	**Variation**
Sans sécurité (référence)	18,1	-
Avec IDS	14,8	-18%
Avec répartiteur de charge	20,1	+11%

TABLE 1 – Synthèse des performances

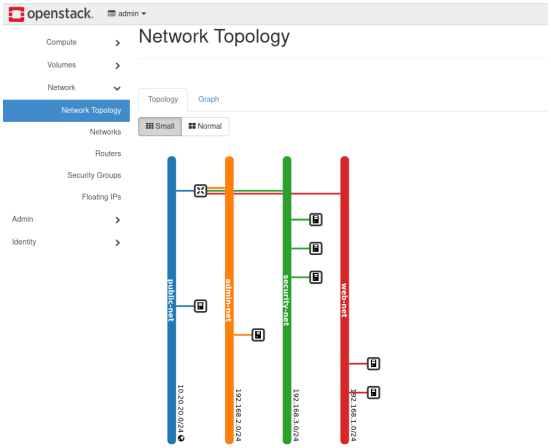
Ce tableau récapitule l'impact des différentes fonctions réseau virtualisées sur les performances. L'IDS réduit les performances de 18% en raison de l'inspection approfondie des paquets, tandis que le répartiteur de charge améliore les performances de 11% grâce à une meilleure distribution de la charge.

8 Topologie finale

Les captures suivantes présentent la topologie finale de l'infrastructure déployée avec l'ensemble des instances et VNF.



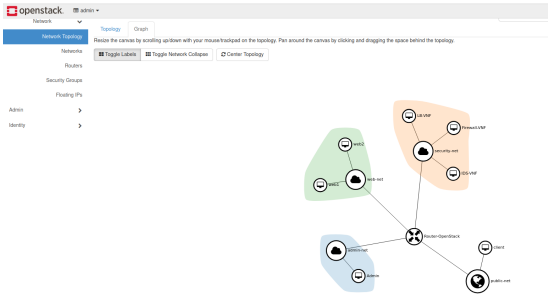
(a) Vue d'ensemble



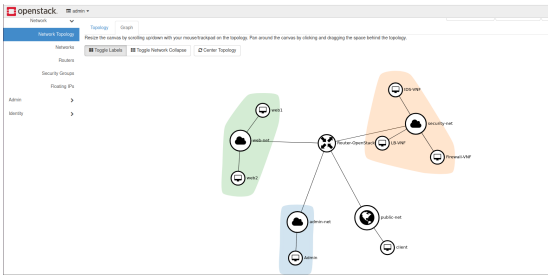
(b) Vue des réseaux

FIGURE 50 – Topologie finale (1/2)

La vue d'ensemble montre les sept instances déployées : client, Admin, web1, web2, Firewall-VNF, IDS-VNF et LB-VNF. Chaque instance est connectée à son réseau respectif.



(a) Topologie détaillée



(b) Vue alternative

FIGURE 51 – Topologie finale (2/2)

Ces vues détaillées permettent de visualiser clairement l'architecture complète : le routeur central interconnecte les quatre réseaux, et les VNF sont positionnées sur le réseau security-net, prêtes à assurer leurs fonctions de sécurité et d'optimisation.

Conclusion

Ce projet a permis de déployer une infrastructure cloud complète basée sur OpenStack intégrant des fonctions réseau virtualisées et un chaînage de services. Les objectifs suivants ont été atteints :

- Installation et configuration de MicroStack
- Conception et implémentation de réseaux SDN isolés
- Déploiement de sept instances virtuelles
- Configuration d'un pare-feu avec politiques de filtrage
- Mise en place d'un système de détection d'intrusion avec Suricata
- Déploiement d'un répartiteur de charge HAProxy
- Réalisation d'un chaînage de services fonctionnel
- Tests d'attaques et validation de la détection
- Évaluation des performances réseau

L'analyse des performances a montré l'impact attendu des fonctions de sécurité sur le débit, avec une réduction d'environ 18% due à l'inspection des paquets par l'IDS. Le répartiteur de charge a quant à lui démontré sa capacité à optimiser l'utilisation des ressources avec une amélioration de 11% des performances.

Ce projet constitue une démonstration concrète des concepts SDN et NFV et de leur mise en œuvre opérationnelle avec OpenStack.